

Cafe Sales Project

How the Messy Sales Data Was Cleaned, Step by Step

1. What This Document Is About

This project started with a messy, raw export of cafe till receipts: 10,000 rows of sales data with a lot of blanks, typos, and placeholder text like "ERROR" and "UNKNOWN" sitting where real numbers should have been.

This document walks through, in plain language, exactly what was wrong with the data and exactly what was done to fix it in the order it was actually done, with the real before-and-after counts at each step. It is written so that someone with no data background can follow the reasoning.

2. What Was Wrong With the Raw Data

Before touching anything, the raw file was inspected column by column. Two separate problems showed up:

2.1 Fake "missing" markers

Some blank cells weren't actually blank; they contained the literal words "ERROR" or "UNKNOWN" typed into the cell instead of a real value. A computer does not automatically recognise these as missing information; it treats them as if they were real answers, which would have quietly broken any totals or averages calculated later.

2.2 Genuinely blank cells

Once "ERROR" and "UNKNOWN" were converted into proper blanks, the true scale of the missing data became clear:

Column	Blank cells	% of all 10,000 rows
Item (what was sold)	969	9.7%
Quantity	479	4.8%
Price Per Unit	533	5.3%
Total Spent	502	5.0%
Payment Method	3,178	31.8%
Location (in-store or takeaway)	3,961	39.6%
Transaction Date	460	4.6%

Payment Method and Location were missing on close to a third and two-fifths of all rows. That much missing data in only those two columns points to a problem at the till itself; most likely those two fields weren't always being recorded, rather than random accidents scattered evenly across the whole file.

3. How the Missing Data Was Fixed, Step by Step

Nothing was filled in blindly. Each column was handled differently depending on whether there was a reliable way to work out the true answer, and every step below was checked against the data before moving to the next one.

Step 1 — Turn the fake blanks into real blanks

Every "ERROR" and "UNKNOWN" value, in every column, was converted into a proper blank. This had to happen first, so that every later step was working with an honest picture of what was actually missing.

Step 2 — Tell the computer what kind of data each column holds

Every column had been loaded as plain text, including the ones that are really numbers (Quantity, Price, Total Spent) or a date (Transaction Date). These were converted to their correct types. Without this step, it isn't possible to add up revenue, average an order value, or work with dates at all the computer would just see them as words, not numbers.

Step 3 — Work out missing prices from the item sold

Every menu item at this cafe has one fixed price (Coffee is always \$2.00, Salad is always \$5.00, and so on). So wherever the price was missing, but the item name was known, the item's normal price was used to fill it in.

This recovered 479 of the 533 missing prices, cutting missing Price Per Unit down to 54.

Step 4 — Work out missing item names from the price

This works in reverse: if a row is missing the item name but the price is known, and that exact price only ever belongs to one menu item, the item name can be filled in with total confidence. For example, if \$5.00 is only ever charged for Salad, then a missing item at \$5.00 must be Salad.

The important caution here: some prices are shared by more than one item (for example, two items might both cost \$3.00). Those cases were deliberately left alone rather than guessed, because there was no way to know which of the two items it actually was. Guessing would have introduced wrong answers dressed up as recovered ones.

This recovered 468 of the 969 missing item names, cutting missing Item down to 501.

Step 5 — Work out missing quantities and totals from each other

Total Spent is always Quantity multiplied by Price. So if any two of those three numbers were known, the third could be calculated exactly rather than guessed; no assumptions needed.

This recovered the remaining calculable rows, cutting missing Quantity down to 23 and missing Total Spent down to 23.

Step 6 — Fill Payment Method and Location using the existing pattern

These two columns had no logical link to any other column, so there was no way to calculate the true answer. A statistical approach was needed instead, and two options were weighed:

- Fill every blank with the single most common answer. Rejected: with a third to two-fifths of the column missing, this would have artificially forced a huge chunk of the data into one category (e.g., suddenly "Cash" would look far more popular than it really is).
- Fill each blank by randomly drawing from the existing mix of answers, in the same proportions they already appear. Used instead: if the known data is roughly 40% Cash, 35% Credit Card and 25% Digital Wallet, the blanks get filled in that same 40/35/25 split so the overall pattern of the column is preserved rather than distorted.

This brought both Payment Method and Location down to zero missing values.

Step 7 — Fill the remaining missing item names the same way

After Steps 3–5, 501 item names still couldn't be worked out from price alone. The same proportional-fill approach from Step 6 was applied to these too, bringing Item down to zero missing values.

Step 8 — Go back and recover the last missing prices

Once every item name was known (after Step 7), Step 3's logic could be run a second time. The remaining 54 missing prices could now all be filled in using the item's known price, bringing Price Per Unit down to zero missing values as well.

This second pass matters: it means every single missing price was ultimately recovered with certainty — none had to be dropped or guessed.

Step 9 — Remove what genuinely could not be recovered

After every possible fix above, three columns still had a small number of blanks left with no remaining way to work them out: Quantity (23 rows), Total Spent (23 rows), and Transaction Date (460 rows). For each of these:

- A missing date cannot be guessed; there is no logical or statistical basis for assigning a transaction to one day over another.
- Quantity and Total Spent are core numbers used in every revenue calculation. Filling them with placeholder text like "Unknown" would have converted the entire column from numbers back into text, which would break every sum and average calculated from it afterwards.

Rows with any of these three still missing were removed. Because a handful of rows were missing more than one of these three fields at once, the total number of rows removed (486) is smaller than the sum of the three individual counts ($23 + 23 + 460 = 506$).

Note on a discrepancy found while reviewing this project

The earlier version of this documentation stated that 531 rows (5.31%) were removed, leaving a final dataset of 9,469 rows. Running the actual cleaning notebook end-to-end produces a different, smaller number: 486 rows removed (4.86%), leaving 9,514 rows.

That is the number this document now uses, since it comes directly from the notebook rather than a summary. Worth checking: the spreadsheet file that the Power BI dashboard was actually built from currently has 9,469 rows, meaning the dashboard was built on a different version of the cleaned file than the one this notebook currently produces. This should be reconciled before treating the dashboard's totals as final, since a roughly half-a-percent difference in row count will shift Total Revenue and the other KPI cards slightly.

4. Double-Checking the Clean-Up

4.1 Checking for repeated rows

The cleaned data was checked for exact duplicate rows, the same transaction accidentally appearing twice. None were found.

4.2 Checking for unusually large transactions

A standard statistical check (it flags any value sitting far outside the normal range of the rest of the data) was run on Quantity, Price, and Total Spent. Quantity and Price came back completely normal. Total Spent flagged 259 transactions, about 2.7% of the cleaned data, as unusually high.

Rather than deleting these automatically, each one was looked at individually. Every single flagged transaction turned out to be the same thing: 5 units of Salad at \$5.00 each, for a total of \$25.00 — which is the highest amount any single order at this cafe could ever legitimately reach (the most expensive item, bought the maximum number of times). None of the 259 were impossible or broken; they were simply large, genuine orders.

Decision: all 259 were kept. A value looking unusual compared to the rest of the data is not the same thing as a value being wrong.

5. The Cleaned Dataset, in Summary

Measure	Value
Rows in the raw file	10,000
Rows removed (unrecoverable)	486 (4.86%)
Rows in the final cleaned file	9,514
Duplicate rows found	0
Time period covered	1 January 2023 – 31 December 2023
Columns with missing data remaining	0

Every column now holds the correct type of data (numbers as numbers, dates as dates), every column is fully filled in, and the file is ready to be used for reporting or dashboarding without any further clean-up.

6. What the Cleaned Data Shows

- Best sellers: Coffee, Salad, and Cookie are the most frequently sold items; Smoothie and Sandwich sell the least often. (Note: this is a count of how often each item was sold it is a separate question from which item brings in the most money, which depends on price as well.)
- Order size: the average transaction is worth about \$8.92, with individual orders ranging from \$1.00 up to the \$25.00 maximum described in Section 4.2.
- Sales over time: daily and monthly sales are steady across the whole year, without a clear growth trend or an obvious busy season.